Chapitre 03 — Écrire son premier script

Objectif Transformer quelques commandes en un programme que tu peux relancer, lire et vérifier.

1. Un script est un fichier texte

Un script Bash contient les commandes que tu aurais pu taper dans le terminal. Bash les lit dans l'ordre. Tu peux ainsi refaire la même tâche sans recopier les commandes.

Le cycle d'un petit script



Écrire, vérifier puis exécuter

Commence dans un dossier de travail :

```
mkdir -p ~/atelier-bash/scripts
cd ~/atelier-bash/scripts
```

2. Créer le fichier avec nano

Ouvre l'éditeur :

interactif: editeur

```
nano accueil.sh
```

Dans l'éditeur, saisis le contenu suivant. Ne recopie pas les marques de bloc de code.

fichier: accueil.sh

```
#!/usr/bin/env bash

# Un nom peut etre fourni au lancement.
prenom="${1:-visiteur}"
station="Orion"

echo "Bienvenue $prenom sur $station."
```

Ligne	Rôle
`#!/usr/bin/env bash`	Demande d'utiliser Bash pour l'exécution directe

Ligne	Rôle
`# Un nom…`	Commentaire : Bash l'ignore, il s'adresse à qui lit le fichier
`prenom="${1:-visiteur}"`	Prend le premier argument fourni au script, ou `visiteur` s'il n'y en a pas
`station="Orion"`	Range une valeur fixe
`echo ...`	Produit le message

La troisième ligne est la plus chargée : `$1` désigne le premier argument et `${1:-visiteur}` fournit une valeur de secours, comme le `${nom:-defaut}` du chapitre 02. Le §5 y revient en détail — pour l'instant, recopie-la telle quelle.

La première ligne s'appelle le **shebang**. Nous écrivons `#!/usr/bin/env bash` plutôt que `#!/bin/bash` : `env` va chercher Bash là où il est réellement installé, ce qui rend le script utilisable sur des systèmes où le chemin diffère.

Dans nano : **Ctrl+O**, puis **Entrée** pour enregistrer ; **Ctrl+X** pour quitter. Les indications `^O` et `^X` de nano signifient « Ctrl+O » et « Ctrl+X ».

Fichier créé sous Windows Si tu vois une erreur mentionnant `bash\r`, le fichier a été enregistré avec des fins de ligne Windows (CRLF). Un script Linux doit être enregistré en **LF**. En travaillant avec nano dans Debian, tu évites ce problème.

3. Vérifier avant de lancer

```
bash -n accueil.sh
```

`-n` vérifie la syntaxe **sans exécuter** les commandes. Si rien ne s'affiche et que la commande réussit, Bash n'a pas trouvé d'erreur de syntaxe. Cela ne garantit pas que le programme fait ce que tu souhaites : il faut aussi le tester.

Lance-le ensuite en indiquant l'interpréteur :

```
bash accueil.sh Mila
```

Résultat : `Bienvenue Mila sur Orion.`. Dans ce mode, le fichier n'a pas besoin d'être exécutable.

4. Exécuter directement le script

```
chmod +x accueil.sh
./accueil.sh Mila
```

- `chmod +x` ajoute le droit d'exécution.
- `./accueil.sh` désigne le fichier dans le dossier courant.
- `Mila` est un argument transmis au script.

Le `./` n'est pas décoratif : le dossier courant n'est généralement pas dans `PATH`. Pour un script stocké ailleurs, indique son chemin.

Noter aussi ce qui **n'est pas** nécessaire : aucun `sudo` ici. Tu es propriétaire de ton dossier personnel, donc tu peux y créer un fichier, le rendre exécutable et le lancer sans droits administrateur. Un script qui réclame `sudo` pour travailler dans `~` est le signe qu'il se trompe de dossier.

5. Les arguments : des valeurs fournies au départ

Dans le script, `$1` est le premier argument. `${1:-visiteur}` donne une valeur par défaut si ce premier argument est absent ou vide.

```
./accueil.sh
./accueil.sh "Mila Vega"
```

Résultats :

```
Bienvenue visiteur sur Orion.
Bienvenue Mila Vega sur Orion.
```

Les guillemets autour de `Mila Vega` conservent un seul argument.

Variable spéciale	Signification
`$0`	Nom utilisé pour lancer le script
`$1`, `$2`	Premier et deuxième arguments
`$#`	Nombre d'arguments
`"$@"`	Tous les arguments, en conservant leurs limites

Pour observer les arguments, crée un second fichier :

fichier: arguments.sh

```
#!/usr/bin/env bash
echo "Nombre : $#"
printf 'Argument : %s\n' "$@"
```

```
bash arguments.sh "Mila Vega" pilote
```

```
Nombre : 2
Argument : Mila Vega
Argument : pilote
```

`"$@"` ne recoupe pas `Mila Vega` en deux mots : c'est bien un seul argument.

Une ligne de format, deux lignes affichées : `printf` **réapplique son format tant qu'il reste des arguments à consommer**. C'est le seul endroit de ce chapitre où `printf` rend un service qu'`echo` ne sait pas rendre — `echo "Argument : $@"` n'afficherait qu'une seule ligne, avec les deux valeurs à la suite. Partout ailleurs, `echo` suffit.

6. Le code de retour

Une commande fournit un **code de retour**. `0` signifie qu'elle a réussi selon ses propres règles. Une valeur différente de zéro indique un autre résultat : erreur, absence de correspondance pour `grep`, etc.

```
bash accueil.sh Mila
code=$?
echo "Code : $code"
```

`$?` donne le code de la **dernière commande**. Ici, le script réussit et le code est `0`. Il faut le lire tout de suite, avant qu'une autre commande le remplace.

Observe un échec sans créer de fichier :

```
ls dossier-inexistant-orion 2>/dev/null
code=$?
echo "Code apres echec : $code"
```

Le code est différent de zéro. `2>/dev/null` masque le message d'erreur ; il ne transforme pas l'échec en réussite. Nous verrons comment réagir au résultat dans le chapitre suivant.

7. Comprendre un comportement inattendu

```
bash -x accueil.sh Mila
```

`-x` affiche les commandes après les expansions. Cela aide à comprendre les valeurs réellement utilisées. Les lignes de trace et le résultat normal peuvent apparaître ensemble.

> Trace et données personnelles Une trace peut afficher les valeurs des variables. Utilise les données fictives du cours avant de partager une capture.

À retenir

- Le shebang choisit l'interpréteur lors de l'exécution directe ; nous utilisons `#!/usr/bin/env bash`.
- `bash -n` vérifie la syntaxe ; les essais vérifient le comportement ; `bash -x` montre les valeurs réellement utilisées.
- `chmod +x` donne le droit d'exécution, `./` indique le fichier du dossier courant : les deux sont nécessaires.
- Les arguments se lisent avec `$1`, `$#` et `"$@"` ; `${1:-defaut}` prévoit le cas où l'argument est absent.
- Protège les arguments avec des guillemets.
- `$?` donne le code de retour de la dernière commande : `0` pour une réussite, autre chose sinon. Lis-le tout de suite.
- Un script n'a pas besoin de `sudo` pour travailler dans ton dossier personnel.

À toi : combien d'arguments reçoit `bash arguments.sh "poste radio" Orion` ? Pourquoi `chmod +x` ne suffit-il pas à lancer un fichier ? Que vaut `$?` juste après une commande qui a échoué ?

Passer au TP